COMUNICA SERTÃO — ROTEIRO DE APRESENTAÇÃO (15 SLIDES)

Guia Rápido de Fala Slide por Slide & Respostas Técnicas para a Banca

IFRS - Campus Sertão
Tecnologia em ADS
(2026)

Autor: Fabricio Cardoso de Abreu **Orientadora:** Caroline Guterres Silva **Foco:** Arquitetura Back-End, APIs & Dados

1. ROTEIRO DE FALA SLIDE A SLIDE (15 SLIDES)

SLIDE 01 Capa Oficial (Comunica Sertão: Arquitetura Back-End)

> "Olá a todos. Meu nome é Fabricio Cardoso de Abreu, sob orientação da professora Caroline Guterres Silva, e apresento o desenvolvimento da arquitetura de Back-End do projeto Comunica Sertão."

SLIDE 02 Contexto e Justificativa da Solução (Gargalos e Motivação Técnica)

> "Sertão possui uma área rural extensa e a prefeitura recebia demandas de forma desarticulada, sem auditoria nem protocolos. Projetamos uma API centralizada em Python e FastAPI com SQLite local: rápida, com integridade ACID, dados sob custódia do município e custo zero de licenças."

SLIDE 03 Objetivos de Desenvolvimento da API (Metas Estruturantes)

> "Nossos 4 objetivos estruturantes foram: construir o barramento RESTful, processar os webhooks do WhatsApp em tempo real, garantir persistência relacional das ocorrências e implementar governança de acessos com aprovação de servidores e emissão de protocolos formais."

SLIDE 04 Análise Comparativa de Arquiteturas (Mercado vs. Comunica Sertão)

> "Ao analisar o mercado, vimos que o Fala.BR é monolítico e complexo, enquanto plataformas como Colab.re e Octadesk cobram mensalidades por operador e retêm dados em nuvens privadas. O Comunica Sertão é sob medida: código aberto, baixa latência e total soberania dos dados públicos."

SLIDE 05 Requisitos do Sistema (RFs Funcionais e RNFs de Qualidade)

> "Definimos o que a API faz nos RFs: recepção de webhooks, emissão de protocolos, consulta em tempo real e controle de acessos. E garantimos qualidade nos RNFs: resposta em menos de 200 milissegundos, integridade ACID no banco, validação com Pydantic e segurança institucional com e-mail corporativo."

SLIDE 06 Visão Geral da Arquitetura (Camadas Orientadas a Serviços)

> "Nossa arquitetura opera em camadas: o cidadão no WhatsApp e o servidor no painel atingem o FastAPI Gateway com Uvicorn ASGI; as requisições passam pelos esquemas Pydantic e middlewares CORS, acionam o Bot Service e o módulo de autenticação, e são persistidas via SQLAlchemy no SQLite."

SLIDE 07 **Mapeamento de Casos de Uso (UML)**

> "Modelamos as interações entre os dois atores centrais: o cidadão interage pelo WhatsApp registrando chamados com fotos e consultando protocolos; o servidor acessa via rotas REST autenticadas para triar demandas, mudar status no Kanban e aprovar novos operadores."

SLIDE 08 **Modelo de Dados Relacional (DER no SQLite)**

> "O banco relacional estrutura as entidades Cidadão, Ocorrência, Funcionário e Histórico. Destaco o relacionamento 1 para N entre cidadão e chamados, e a rastreabilidade garantida pela tabela de histórico de status com chaves estrangeiras."

SLIDE 09 **Pilha Tecnológica: FastAPI, Python 3 e Uvicorn (Execução)**

> "O FastAPI opera sobre o padrão ASGI com Python 3 e servidor Uvicorn. Essa combinação assíncrona com async/await viabiliza concorrência massiva de mensagens com tempos médios de resposta abaixo de 50 milissegundos e consumo mínimo de memória."

SLIDE 10 **Pilha Tecnológica: SQLite, SQLAlchemy e Pydantic (Dados e Validação)**

> "O SQLite local elimina a complexidade de gerenciar servidores de banco externos, enquanto o SQLAlchemy ORM assegura transações ACID. O Pydantic valida os contratos das rotas, bloqueando dados corrompidos ou maliciosos antes de qualquer gravação."

SLIDE 11 **Implementação: Documentação Interativa OpenAPI (/docs)**

> "A API gera a documentação Swagger UI nativamente na rota /docs. Isso nos permitiu testar em tempo real as chamadas de login institucional, listagem de chamados, simulador de chat e endpoints de homologação de forma padronizada."

SLIDE 12 **Implementação: Motor Conversacional do Bot e Protocolos**

> "Implementamos a máquina de estados que processa os webhooks. O bot conduz o cidadão por opções simples, coleta fotos e local, grava no SQLite e gera o protocolo anual indexado (ex: CMS-2026-0004) para acompanhamento."

SLIDE 13 **Implementação: Controle de Acessos e Governança**

> "Para segurança institucional, a API exige e-mails oficiais @sertao.rs.gov.br. Novos servidores são gravados como 'Pendente' e sofrem bloqueio HTTP 403 até que um dos três perfis Administradores homologados aprove o acesso no painel."

SLIDE 14 **Implementação: Gestão de Ocorrências e Mídias**

> "O endpoint /upload/foto valida extensões de imagens (JPG, PNG, WEBP), gera nomes únicos com UUID e serve os arquivos estáticos. Já as rotas de mutação de status executam commits atômicos, sincronizando imediatamente o Kanban do painel."

SLIDE 15 **Conclusão e Trabalhos Futuros**

> "Concluímos uma API robusta, leve e de custo zero para Sertão. Como próximos passos, faremos a hospedagem em nuvem com SSL, a ativação do webhook oficial na Meta Cloud API com o chip da prefeitura e o envio automático de notificações no WhatsApp do munícipe a cada avanço do protocolo. Obrigado!"

2. RESPOSTAS RÁPIDAS PARA A BANCA

1. "Por que SQLite e não um banco como PostgreSQL?"

Resposta: Para a escala do município (5.500 habitantes), o SQLite é rápido, atômico (ACID) e não consome memória em background. Além disso, pelo uso do SQLAlchemy ORM, se houver expansão futura, migrar para PostgreSQL exige apenas alterar uma string de conexão.

2. "Como funciona o fluxo de aprovação de novos servidores?"

Resposta: O cadastro exige e-mail @sertao.rs.gov.br e é salvo com status "Pendente". A API retorna HTTP 403 em tentativas de login até que um dos três Administradores execute o endpoint PATCH para homologar a conta.

3. "O back-end suporta múltiplos munícipes enviando mensagens ao mesmo tempo?"

Resposta: Sim. O FastAPI e o Uvicorn usam o padrão ASGI com async/await, processando as requisições de webhook de modo assíncrono e não bloqueante, com latência média abaixo de 50ms.